

Sesión 3: Protección de APIs y Gestión Segura de Secretos

Módulo IV · Seguridad Aplicada a Entornos Java & .NET

DURACIÓN: 3 H 20 MIN

NIVEL SENIOR / ARQUITECTURA



Objetivos de la sesión

Al finalizar esta sesión, los participantes serán capaces de diseñar e implementar políticas de protección robustas para APIs en entornos de producción, y de gestionar credenciales y secretos de forma segura mediante herramientas especializadas del ecosistema cloud.

Proteger APIs

Rate limiting, CORS, SSL/TLS y cabeceras de seguridad en entornos reales

APIs Públicas

Gateways, WAF y throttling para exposición controlada y segura

Secretos

Vault, AWS Secrets Manager y Azure Key Vault: principios y configuración

Taller

Aplicación práctica de políticas y externalización segura de credenciales

Agenda de la sesión

01

Seguridad en APIs

Rate Limiting · CORS · SSL/TLS · Security Headers

03

Gestión de Secretos

Principios · Vault · AWS Secrets Manager · Azure Key Vault

02

Seguridad en APIs Públicas

API Gateways · WAF · Throttling avanzado

04

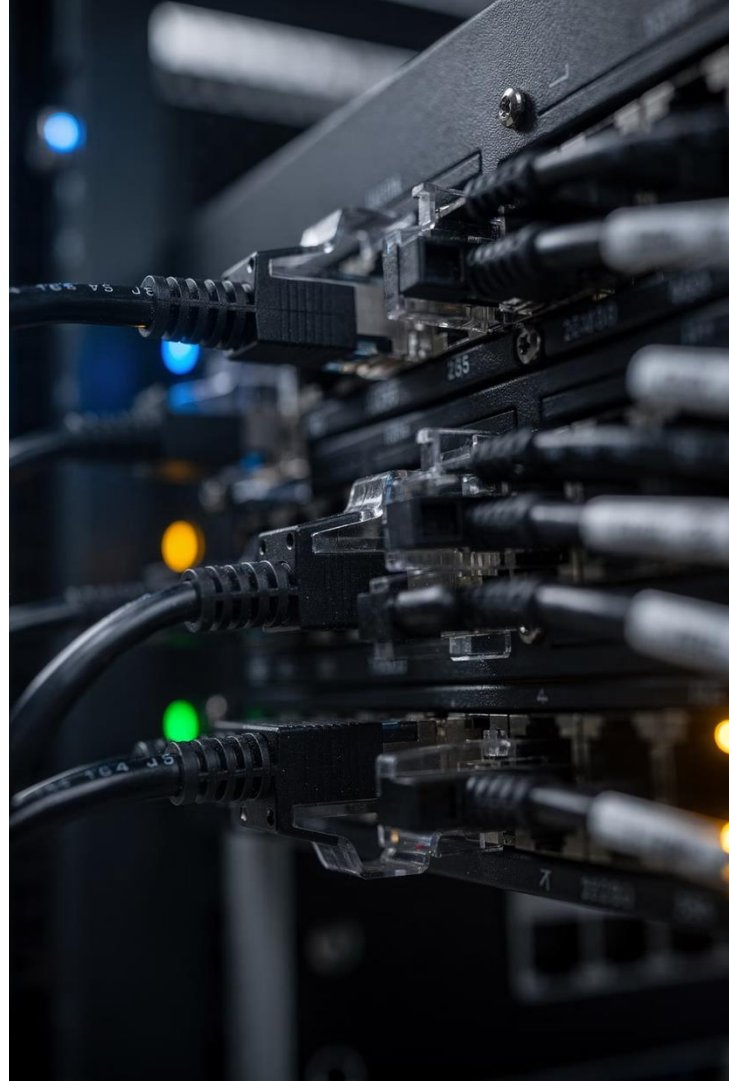
Taller Práctico

Configuración de políticas · Externalización de credenciales

PARTE 1

Seguridad en APIs

Fundamentos de protección para APIs backend en entornos Java y .NET



¿Por qué es crítica la seguridad en APIs?

Las APIs son la superficie de ataque más expuesta en aplicaciones modernas. Según el informe OWASP API Security Top 10, la mayoría de brechas de datos en 2023 tuvieron como vector de entrada una API mal protegida. A diferencia de las interfaces web, las APIs rara vez cuentan con protecciones implícitas del navegador, por lo que cada capa de seguridad debe ser diseñada e implementada de forma explícita.

 Una API sin protección adecuada equivale a una puerta trasera abierta en tu sistema.

Superficies de riesgo

- Endpoints no autenticados expuestos
- Ausencia de límites de peticiones
- Cabeceras HTTP inseguras por defecto
- Certificados caducados o mal configurados
- CORS permisivo sin restricciones de origen

Rate Limiting: Fundamentos

El **rate limiting** controla el número de peticiones que un cliente puede realizar a una API en un intervalo de tiempo determinado. Es la primera línea de defensa contra ataques de fuerza bruta, scraping masivo y denegación de servicio (DoS).

Fixed Window

Ventana fija de tiempo (ej. 100 req/min). Sencillo pero susceptible a ráfagas en los bordes del intervalo.

Sliding Window

Ventana deslizante que suaviza el tráfico. Más preciso y resistente a ataques de borde.

Token Bucket

Acumulación de tokens que permite ráfagas controladas. Muy usado en AWS y GCP.

Leaky Bucket

Procesa peticiones a tasa constante independientemente del volumen de entrada. Ideal para suavizar picos.

Rate Limiting en Spring Boot

Con **Bucket4j** integrado en Spring Boot, podemos aplicar rate limiting por endpoint, por usuario o por clave de API, usando Redis o en memoria.

```
// Dependencia: bucket4j-spring-boot-starter
@Bean
public Bucket crearBucket() {
    Bandwidth limite = Bandwidth.classic(
        100, Refill.greedy(100, Duration.ofMinutes(1))
    );
    return Bucket.builder().addLimit(limite).build();
}

// En el filtro HTTP:
if (!bucket.tryConsume(1)) {
    response.setStatus(HttpStatus.TOO_MANY_REQUESTS.value());
    response.addHeader("Retry-After", "60");
    return;
}
```

 Devuelve siempre el código 429 Too Many Requests con la cabecera Retry-After para que los clientes implementen backoff exponencial.

Rate Limiting en ASP.NET Core 7+

Desde .NET 7, el middleware de rate limiting es nativo y no requiere dependencias externas. Permite políticas declarativas por ruta, usuario o IP con poca configuración.

```
// Program.cs
builder.Services.AddRateLimiter(options => {
    options.AddFixedWindowLimiter("api-policy", o => {
        o.Window = TimeSpan.FromMinutes(1);
        o.PermitLimit = 100;
        o.QueueProcessingOrder = QueueProcessingOrder.OldestFirst;
        o.QueueLimit = 10;
    });
    options.RejectionStatusCode = 429;
});

app.UseRateLimiter();

// En el controlador:
[EnableRateLimiting("api-policy")]
[HttpGet("datos-sensibles")]
public IActionResult ObtenerDatos() { ... }
```

CORS: Control de Acceso entre Orígenes

CORS (*Cross-Origin Resource Sharing*) define qué dominios externos tienen permiso para consumir tu API. Una política CORS demasiado permisiva (*) convierte la API en vulnerable a ataques CSRF y robo de credenciales desde scripts maliciosos.

Configuración insegura

```
// NUNCA en producción
cors.allowedOrigins("*");
cors.allowCredentials(true);
// Combinación inválida y peligrosa
```

Permitir cualquier origen con credenciales expone tokens de sesión a sitios maliciosos.

Configuración segura

```
cors.allowedOrigins(
  "https://app.midominio.com",
  "https://admin.midominio.com"
);
cors.allowedMethods("GET", "POST");
cors.allowCredentials(true);
cors.maxAge(3600L);
```

CORS en ASP.NET Core: Configuración por Entorno

En .NET es recomendable definir políticas CORS con nombre y aplicarlas selectivamente a los controladores que las necesiten, en lugar de una política global permisiva.

```
// Program.cs
builder.Services.AddCors(options => {
    options.AddPolicy("PoliticaProduccion", policy => {
        policy.WithOrigins("https://app.midominio.com")
            .WithMethods("GET", "POST", "PUT")
            .WithHeaders("Authorization", "Content-Type")
            .AllowCredentials()
            .SetPreflightMaxAge(TimeSpan.FromSeconds(3600));
    });
    options.AddPolicy("PoliticaDesarrollo", policy => {
        policy.WithOrigins("http://localhost:3000")
            .AllowAnyMethod().AllowAnyHeader();
    });
});

app.UseCors(env.IsProduction()
    ? "PoliticaProduccion"
    : "PoliticaDesarrollo");
```

SSL/TLS: Cifrado en Tránsito

Todo tráfico de APIs en producción debe viajar cifrado. TLS 1.3 es el estándar actual: más rápido y más seguro que sus predecesores, eliminando negociaciones innecesarias y cifrados obsoletos.



Configuración TLS: Mejores Prácticas

Versiones permitidas

Deshabilitar TLS 1.0 y 1.1 en todos los entornos. Permitir únicamente TLS 1.2 (legacy) y TLS 1.3. Configúralo a nivel de servidor (Nginx, IIS, Apache) y a nivel de aplicación.

Cipher suites

Usar únicamente suites con Perfect Forward Secrecy (ECDHE).
Deshabilitar RC4, DES, 3DES y MD5. Herramienta: `testssl.sh` o SSL Labs para auditoría.

Gestión de certificados

Automatizar renovación con Let's Encrypt + Certbot o con el servicio ACM de AWS. Configurar alertas de expiración con 30 días de antelación.

HSTS

Habilitar HTTP Strict Transport Security con `max-age=31536000; includeSubDomains; preload` para forzar HTTPS en el navegador.

Security Headers: La Línea de Defensa Invisible

Las cabeceras de seguridad HTTP son instrucciones que el servidor envía al cliente (navegador u otro consumidor) para indicar qué comportamientos están permitidos. Son baratas de implementar y enormemente efectivas contra XSS, clickjacking y sniffing de contenido.

Cabecera	Propósito	Valor recomendado
Content-Security-Policy	Previene XSS limitando fuentes de contenido	default-src 'self'
X-Frame-Options	Evita clickjacking en iframes	DENY
X-Content-Type-Options	Impide MIME sniffing	nosniff
Referrer-Policy	Controla información de referencia	strict-origin-when-cross-origin
Permissions-Policy	Restringe APIs del navegador	camera=(), microphone=()

Security Headers en Spring Boot

Spring Security aplica un conjunto básico de cabeceras de seguridad por defecto. Sin embargo, para entornos de producción es necesario personalizar la política CSP y ajustar cada cabecera al contexto de la aplicación.

```
@Configuration
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http)
        throws Exception {
        http.headers(headers -> headers
            .contentSecurityPolicy(csp -> csp
                .policyDirectives(
                    "default-src 'self'; " +
                    "script-src 'self' 'nonce-{nonce}'; " +
                    "img-src 'self' data: https:; " +
                    "frame-ancestors 'none'"))
            .frameOptions(frame -> frame.deny())
            .referrerPolicy(rp -> rp
                .policy(ReferrerPolicyHeaderWriter.ReferrerPolicy
                    .STRICT_ORIGIN_WHEN_CROSS_ORIGIN))
        );
        return http.build();
    }
}
```

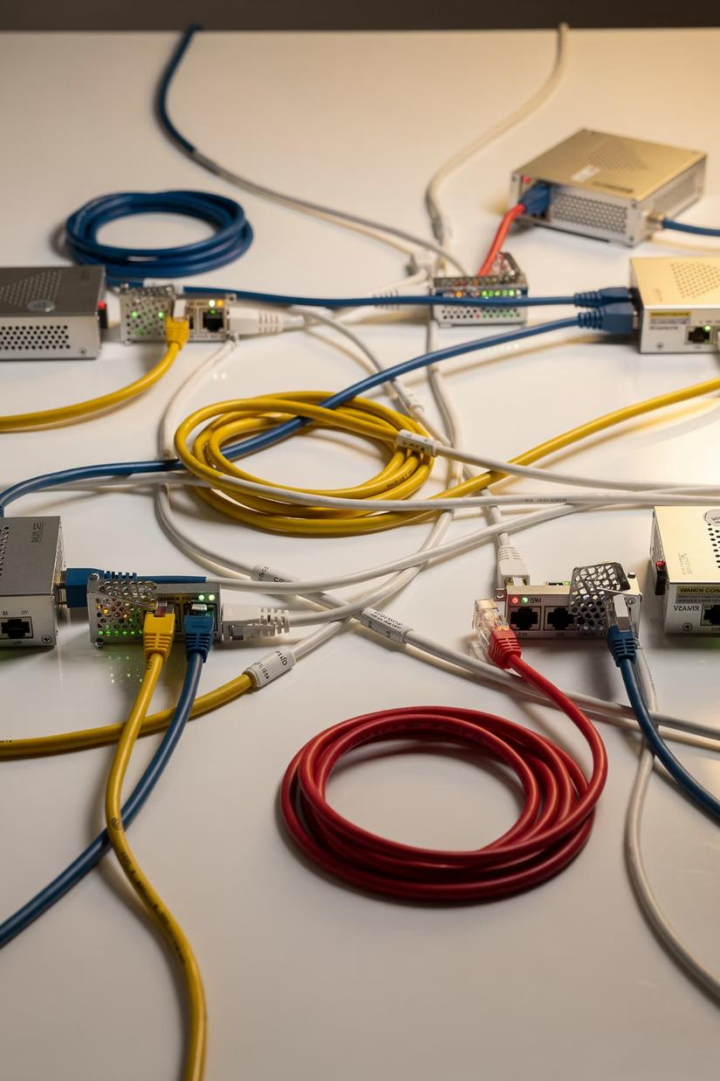
Security Headers en ASP.NET Core con NWebSec

```
// Program.cs – usando middleware NWebSec
app.UseHsts(options => options
    .MaxAge(365).IncludeSubdomains().Preload());

app.UseXContentTypeOptions();
app.UseReferrerPolicy(opts =>
    opts.StrictOriginWhenCrossOrigin());
app.UseXfo(options => options.Deny());

app.UseCsp(opts => opts
    .DefaultSources(s => s.Self())
    .ScriptSources(s => s.Self().Nonce())
    .StyleSources(s => s.Self())
    .ImageSources(s => s.Self().CustomSources("data:"))
    .FrameAncestors(s => s.None())
);
```

✔ Valida tu configuración de cabeceras con **securityheaders.com** después de cada despliegue. El objetivo es obtener una puntuación A o A+.



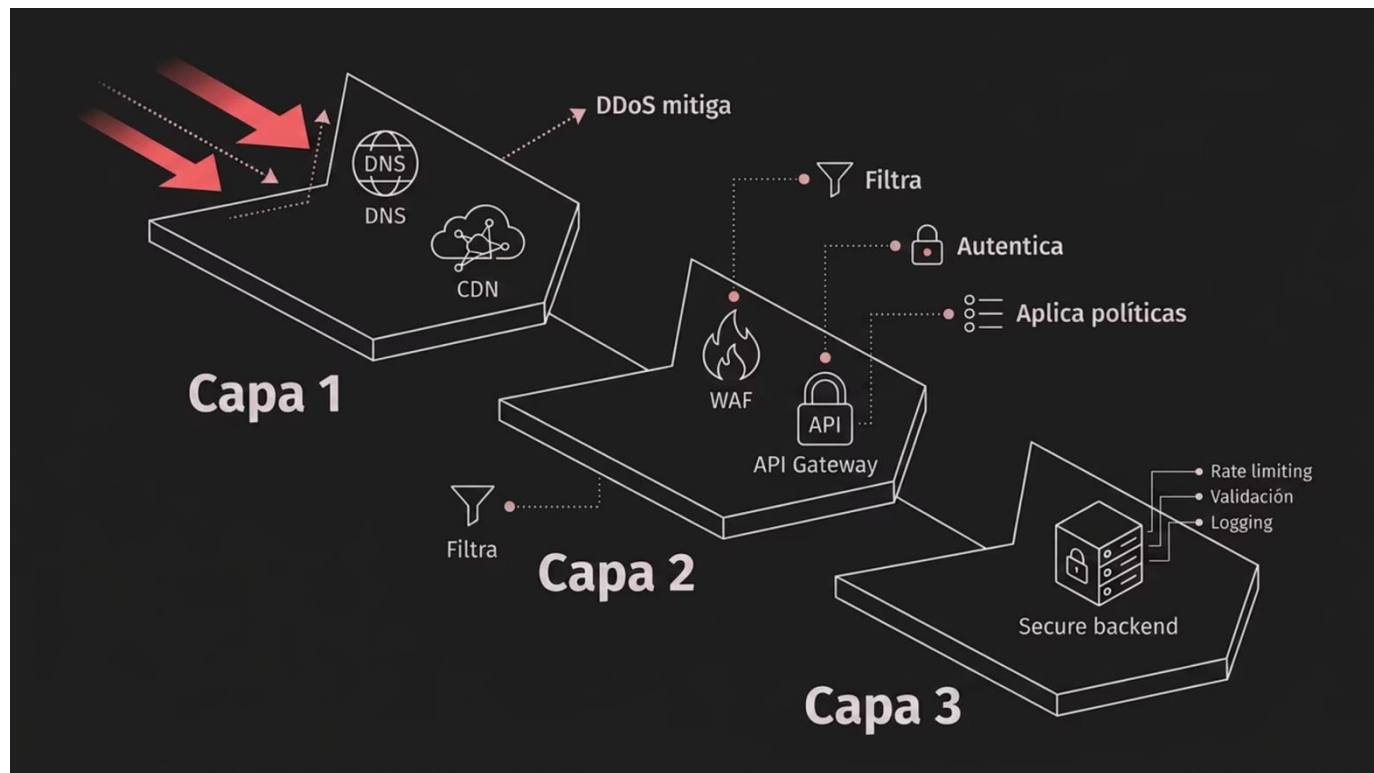
PARTE 2

Seguridad en APIs Públicas

Gateways, WAF y throttling para una exposición controlada y segura al exterior

El Reto de Exponer APIs al Mundo

Cuando una API sale a internet, la superficie de ataque se multiplica exponencialmente. Ya no sólo interactúan clientes internos controlados, sino bots, scrapers, actores maliciosos y consumidores legítimos a escala global. La arquitectura de seguridad debe adaptarse a este contexto con capas de protección que actúen antes de que el tráfico llegue al servicio de negocio.



API Gateways: El Guardián de tu API

Un **API Gateway** es el punto de entrada centralizado que actúa como intermediario entre los clientes y los servicios backend. Concentra funciones transversales de seguridad para que cada microservicio no tenga que implementarlas individualmente.



Autenticación centralizada

Valida tokens JWT, API Keys y OAuth2 antes de que la petición llegue al backend. Un único punto de control reduce la superficie de error.



Políticas de acceso

Define reglas de autorización por ruta, método HTTP o claims del token. Soporta RBAC y ABAC sin lógica en el servicio.



Observabilidad

Logging centralizado, métricas de latencia y tracing distribuido sin instrumentar cada servicio individualmente.

Soluciones de API Gateway: Comparativa

Gateway	Tipo	Rate Limiting	Auth nativo	Ideal para
Kong	OSS / Enterprise	✅ Plugin	JWT, OAuth2, Key Auth	Microservicios, on-prem
AWS API Gateway	Managed	✅ Nativo	Cognito, IAM, Lambda	Ecosistema AWS
Azure API Management	Managed	✅ Políticas XML	AAD, JWT, Key	Ecosistema Azure
NGINX + Lua	Autoalojado	✅ Configurable	Custom middleware	Control total y bajo coste
Apigee (Google)	Managed	✅ Nativo	OAuth2, SAML	Empresas con APIs complejas

Kong Gateway: Configuración de Rate Limiting

Kong usa plugins declarativos. La configuración puede hacerse vía Admin API REST o mediante `deck` (Kong declarative config). Aquí aplicamos rate limiting por consumidor autenticado:

```
# deck.yaml – configuración declarativa
plugins:
  - name: rate-limiting
  config:
    minute: 100
    hour: 1000
    policy: redis # Compartido entre instancias
    redis_host: redis.svc
    redis_port: 6379
    fault_tolerant: true # No bloquear si Redis falla
    hide_client_headers: false
    limit_by: consumer # Por usuario autenticado
  - name: jwt
  config:
    secret_is_base64: false
    claims_to_verify:
      - exp
      - nbf
```

WAF: Web Application Firewall

El **WAF** opera en la capa 7 del modelo OSI inspeccionando el contenido HTTP/HTTPS de cada petición. A diferencia de un firewall de red, entiende la semántica de las peticiones web y puede detectar patrones de ataque como SQLi, XSS o SSRF antes de que lleguen a la aplicación.

Modo de operación

- Detección

Registra ataques sin bloquear. Útil durante la fase de aprendizaje.

- Prevención

Bloquea activamente las peticiones que violan las reglas.

- Reglas gestionadas

OWASP Core Rule Set (CRS) como base, más reglas personalizadas.

Soluciones populares

- **AWS WAF** — integración nativa con CloudFront y ALB
- **Azure Front Door WAF** — gestión de reglas OWASP
- **Cloudflare WAF** — CDN + WAF + DDoS protection
- **ModSecurity** — open source, embed en Nginx/Apache

AWS WAF: Configuración de Reglas Esenciales

```
# Terraform - WAF para API Gateway
resource "aws_wafv2_web_acl" "api_waf" {
  name      = "api-protection"
  scope     = "REGIONAL"

  default_action { allow {} }

  rule {
    name      = "AWSManagedRulesCommonRuleSet"
    priority  = 1
    override_action { none {} }
    statement {
      managed_rule_group_statement {
        vendor_name = "AWS"
        name        = "AWSManagedRulesCommonRuleSet"
      }
    }
    visibility_config {
      cloudwatch_metrics_enabled = true
      metric_name                = "CommonRuleSet"
      sampled_requests_enabled   = true
    }
  }
}
```

③ Combina el CRS de AWS con reglas específicas de SQLi y rate-based rules para bloquear IPs que superen 2.000 peticiones en 5 minutos.

Throttling: Más que un Rate Limit

El **throttling** va más allá del rate limiting: no sólo limita la cantidad de peticiones, sino que gestiona la *calidad del servicio* bajo carga, priorizando tráfico legítimo y degradando graciosamente cuando el sistema está bajo presión. Es un patrón de resiliencia además de seguridad.

1

Throttling por cliente

Límites por API key, usuario o plan de suscripción. Aísla el impacto de un cliente abusivo sobre los demás.

2

Throttling adaptativo

El sistema ajusta los límites dinámicamente en función de la carga del backend. AWS API Gateway lo soporta de forma nativa.

3

Circuit Breaker

Si un servicio downstream falla consistentemente, el gateway deja de enrutar peticiones a él, evitando efecto cascada.



PARTE 3

Gestión de Secretos

Principios, herramientas y patrones para el manejo seguro de credenciales en entornos modernos

El Problema de los Secretos en el Código

El error más común del sector

Según el informe de GitGuardian de 2023, se detectaron **más de 10 millones de secretos** expuestos en repositorios GitHub públicos en un solo año. Contraseñas de base de datos, claves de API, tokens de servicio... todo comprometido por un simple `git push` descuidado.

❌ Un secreto en un repositorio Git, aunque se borre después, puede estar comprometido para siempre por el historial de commits.

¿Qué NO hacer nunca?

- Secretos en `application.properties` / `appsettings.json` en el repo
- Variables de entorno sin cifrar en logs de CI/CD
- Contraseñas hardcodeadas en scripts de despliegue
- Claves compartidas entre entornos (dev = prod)
- Rotar secretos manualmente y sin auditoría

Principios de Gestión Segura de Secretos



Cifrado en reposo y en tránsito

Los secretos deben estar siempre cifrados, tanto almacenados como al ser transmitidos.



Rotación automática

Las credenciales deben tener TTL cortos y rotarse de forma automática sin intervención humana.



Mínimo privilegio

Cada servicio accede únicamente a los secretos que necesita. Nunca un secreto compartido entre servicios diferentes.



Auditoría completa

Cada acceso a un secreto debe quedar registrado: quién, cuándo y desde dónde. Logs inmutables.

Arquitectura de Referencia: Gestión de Secretos

Control de acceso

Políticas IAM y roles limitan lectura por servicio.

Inyección dinámica

Aplicación obtiene secreto en tiempo de ejecución.

Almacenamiento
cifrado

Secreto guardado en
Vault/AWS/Azure con AES-
256.

Rotación automática

Gestor rota credenciales
según TTL y notifica.

El patrón de **secretos dinámicos** elimina por completo la necesidad de que los desarrolladores conozcan o manejen credenciales de producción. La aplicación obtiene credenciales de vida corta directamente del gestor en tiempo de ejecución.



HashiCorp Vault

La solución open source de referencia para gestión de secretos en entornos multi-cloud y on-premises

HashiCorp Vault: Arquitectura y Conceptos

Vault es una plataforma de gestión de secretos que cifra, controla el acceso y audita todo acceso a datos sensibles. Su arquitectura se basa en **motores de secretos** (backends) y un sistema de **políticas HCL**.

Secret Engines

KV (v1/v2), databases, PKI, SSH, AWS, Azure... Cada motor gestiona un tipo de secreto con lógica específica de rotación y generación.

Auth Methods

AppRole, Kubernetes, AWS IAM, LDAP, Token. La aplicación se autentica en Vault con su identidad de plataforma, sin secretos bootstrap.

Políticas HCL

Definen permisos de lectura/escritura por ruta. Granulares al nivel de campo individual de un secreto.

Leases y TTL

Cada secreto tiene un TTL. Al expirar, Vault revoca automáticamente las credenciales en el sistema de origen (ej. base de datos).

Vault: Secretos Dinámicos para Bases de Datos

Los **secretos dinámicos** son credenciales generadas bajo demanda por Vault. Cada solicitud genera un usuario y contraseña únicos en la base de datos, con TTL de 1 hora. Al expirar, Vault los elimina automáticamente.

```
# Habilitar motor de base de datos
vault secrets enable database

# Configurar conexión PostgreSQL
vault write database/config/mi-postgres \
  plugin_name=postgresql-database-plugin \
  connection_url="postgresql://{{username}}:{{password}}@db:5432/app" \
  allowed_roles="app-role" \
  username="vault-admin" \
  password="s3cr3t"

# Definir rol con TTL de 1 hora
vault write database/roles/app-role \
  db_name=mi-postgres \
  creation_statements="CREATE ROLE \"{{name}}\" \
  LOGIN PASSWORD '{{password}}' \
  VALID UNTIL '{{expiration}}';" \
  default_ttl="1h" \
  max_ttl="24h"
```

Vault en Spring Boot con Spring Cloud Vault

```
# bootstrap.yml
spring:
  cloud:
    vault:
      host: vault.miempresa.com
      port: 8200
      scheme: https
      authentication: APPROLE
      app-role:
        role-id: ${VAULT_ROLE_ID}
        secret-id: ${VAULT_SECRET_ID}
      kv:
        enabled: true
        backend: secret
        default-context: mi-servicio
      database:
        enabled: true
        role: app-role
        backend: database
```

- ✔ Spring Cloud Vault renueva automáticamente los leases de los secretos dinámicos antes de que expiren, garantizando continuidad del servicio sin reiniciar la aplicación.

Vault en .NET con VaultSharp

```
// Program.cs
var vaultClientSettings = new VaultClientSettings(
    "https://vault.miempresa.com:8200",
    new AppRoleAuthMethodInfo(
        Environment.GetEnvironmentVariable("VAULT_ROLE_ID"),
        Environment.GetEnvironmentVariable("VAULT_SECRET_ID")
    )
);
var vaultClient = new VaultClient(vaultClientSettings);

// Leer secreto KV v2
var kv2Secret = await vaultClient.V1.Secrets.KeyValue.V2
    .ReadSecretAsync(
        path: "mi-servicio/db-credentials",
        mountPoint: "secret"
    );

string connectionString = kv2Secret.Data.Data["connection_string"].ToString();
```

Integra VaultSharp con el sistema de `IConfiguration` de .NET para que los secretos se inyecten automáticamente en las clases de configuración tipadas.

AWS Secrets Manager

Gestión de secretos nativa en el ecosistema AWS con rotación automática y estrecha integración con servicios gestionados



AWS Secrets Manager: Características Clave



Rotación automática

Lambda functions nativas para rotar credenciales de RDS, Redshift, DocumentDB y secretos personalizados. Configuración sin código con la consola.



Control de acceso IAM

Políticas IAM granulares por secreto, recurso y condición. Integración nativa con KMS para cifrado con claves propias (CMK).



Auditoría con CloudTrail

Cada acceso, modificación o rotación queda registrado en CloudTrail. Integra con CloudWatch Alarms para detectar accesos anómalos.

AWS Secrets Manager en Spring Boot

```
# application.yml – con spring-cloud-aws-starter-secrets-manager
spring:
  config:
    import: "aws-secretsmanager:/prod/mi-servicio/db,
            aws-secretsmanager:/prod/mi-servicio/api-keys"
  datasource:
    url: ${DB_URL}           # Viene del secreto
    username: ${DB_USER}
    password: ${DB_PASS}
```

```
// Acceso programático con SDK v2
SecretsManagerClient client = SecretsManagerClient.builder()
    .region(Region.EU_WEST_1)
    .credentialsProvider(DefaultCredentialsProvider.create())
    .build();

GetSecretValueRequest request = GetSecretValueRequest.builder()
    .secretId("/prod/mi-servicio/db")
    .build();

String secret = client.getSecretValue(request).secretString();
```

AWS Secrets Manager en ASP.NET Core

```
// Program.cs – integración con IConfiguration
builder.Configuration.AddSecretsManager(region: RegionEndpoint.EUWest1,
    configurator: opts => {
        opts.SecretFilter = entry =>
            entry.Name.StartsWith($"{env.EnvironmentName}/MiApp/");
        opts.KeyGenerator = (entry, key) =>
            key.Replace($"{env.EnvironmentName}/MiApp/", "")
                .Replace("/", ":");
        opts.PollingInterval = TimeSpan.FromMinutes(10);
    });

// Los secretos se mapean a la configuración estándar de .NET:
// "Prod/MiApp/ConnectionStrings/Default" → "ConnectionStrings:Default"
```

 Usa `PollingInterval` para que la aplicación detecte rotaciones automáticas sin necesidad de reinicio. Ideal para secretos con TTL cortos.



Azure Key Vault

La solución de Microsoft para gestión de secretos, claves criptográficas y certificados en el ecosistema Azure

Azure Key Vault: Tres Pilares



Secretos

Almacenamiento seguro de cadenas de conexión, contraseñas y tokens. Versionado automático, fechas de expiración y etiquetado para gobierno.



Claves criptográficas

Claves RSA y EC protegidas por HSM (Hardware Security Module). Operaciones criptográficas (cifrar, firmar) sin exponer el material de clave.



Certificados

Ciclo de vida completo de certificados TLS: solicitud, almacenamiento, renovación automática con Let's Encrypt o DigiCert.

El control de acceso se gestiona con **RBAC de Azure** o mediante **Access Policies**. Managed Identity es el mecanismo recomendado para que las aplicaciones accedan sin credenciales explícitas.

Azure Key Vault en Spring Boot

```
# application.yml
azure:
  keyvault:
    uri: https://mi-keyvault.vault.azure.net/
    client-id: ${AZURE_CLIENT_ID}
    tenant-id: ${AZURE_TENANT_ID}
    # En Azure: usar Managed Identity (sin client-secret)
    # En local: usar AZURE_CLIENT_SECRET

# Con Azure Managed Identity (recomendado en producción)
# No se necesita ninguna credencial en el código ni en la config.
# La identidad asignada a la VM/App Service tiene acceso al vault.
```

```
// Acceso programático – Azure SDK para Java
SecretClient secretClient = new SecretClientBuilder()
    .vaultUrl("https://mi-keyvault.vault.azure.net/")
    .credential(new DefaultAzureCredentialBuilder().build())
    .buildClient();

KeyVaultSecret secret = secretClient.getSecret("db-connection-string");
String value = secret.getValue();
```

Azure Key Vault en ASP.NET Core con Managed Identity

```
// Program.cs – la forma más segura: sin credenciales en código
builder.Configuration.AddAzureKeyVault(
    new Uri("https://mi-keyvault.vault.azure.net/"),
    new DefaultAzureCredential() // Usa Managed Identity en Azure
);

// Los secretos del vault (guiones → dos puntos) se mapean:
// "ConnectionStrings--Default" → "ConnectionStrings:Default"

// Acceso programático con tipado fuerte:
public class MiServicio {
    private readonly IConfiguration _config;
    public MiServicio(IConfiguration config) => _config = config;

    public string ObtenerCadenaConexion() =>
        _config["ConnectionStrings:Default"]
        ?? throw new InvalidOperationException("Secreto no encontrado");
}
```

✔ **Managed Identity** es el patrón de oro en Azure: la aplicación no posee ni conoce ninguna credencial. Azure gestiona la identidad y el acceso de forma transparente.

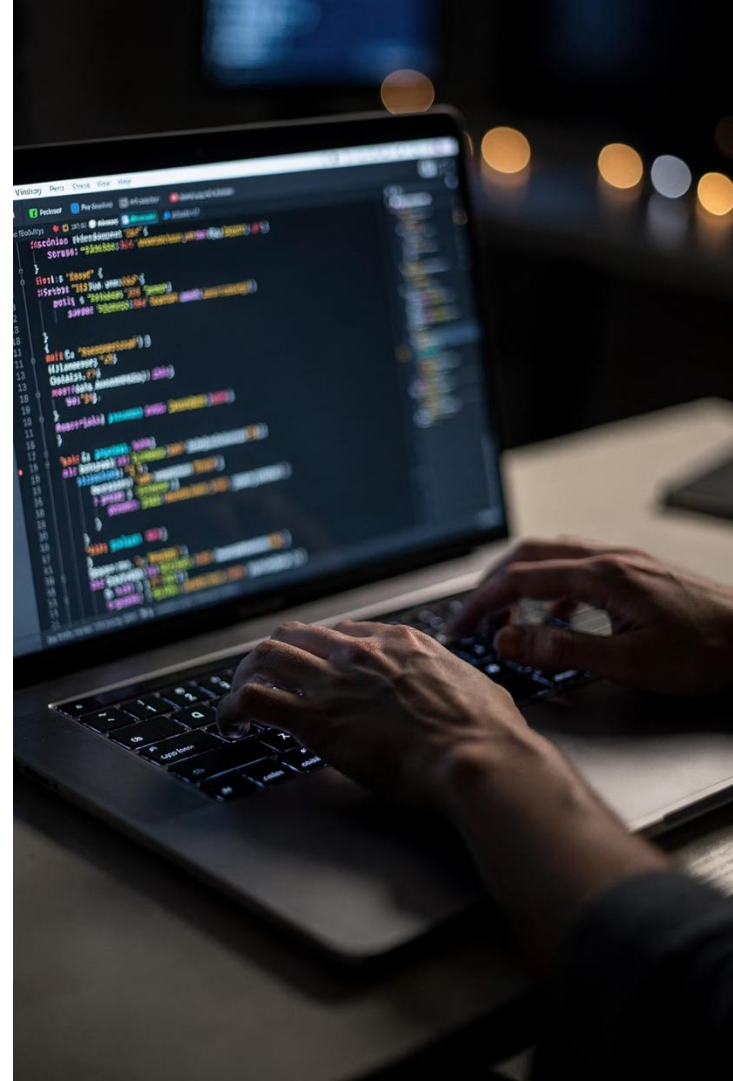
Comparativa: Herramientas de Gestión de Secretos

Característica	HashiCorp Vault	AWS Secrets Manager	Azure Key Vault
Secretos dinámicos	✅ Nativo	❌ Sólo rotación	❌ Sólo rotación
Multi-cloud / on-prem	✅ Sí	⚠️ AWS only	⚠️ Azure only
Rotación automática	✅ Via plugins	✅ Lambda nativa	✅ Con certificados
Integración Kubernetes	✅ Vault Agent / CSI	✅ Secrets Store CSI	✅ Secrets Store CSI
Coste operativo	Alto (autoalojado)	Bajo (managed)	Bajo (managed)
Claves HSM	✅ Vault Enterprise	✅ AWS CloudHSM	✅ Premium tier

PARTE 4

Taller Práctico

Aplicación real de los conceptos: políticas de protección de API y externalización segura de credenciales



Ejercicio 1: Configuración Completa de Protección API

En este ejercicio configuraremos una API Spring Boot con todas las capas de protección estudiadas. El objetivo es que la API resultante supere una auditoría básica de seguridad OWASP.

01

Activar HTTPS y HSTS

Configurar certificado TLS local con `mkcert` y habilitar HSTS en Spring Security

03

Security Headers completos

CSP con nonce, X-Frame-Options, CORS restrictivo por entorno

02

Rate Limiting con Bucket4j + Redis

100 req/min por usuario autenticado; 20 req/min para endpoints públicos

04

Validación con SecurityHeaders.com

Desplegar en local y verificar puntuación A+ en el scanner

Ejercicio 1: Código Base de Referencia

```
@Configuration
@EnableWebSecurity
public class ApiSecurityConfig {

    @Bean
    public SecurityFilterChain chain(HttpSecurity http) throws Exception {
        return http
            .requiresChannel(c -> c.anyRequest().requiresSecure())
            .headers(h -> h
                .httpStrictTransportSecurity(hsts -> hsts
                    .includeSubDomains(true).maxAgeInSeconds(31536000).preload(true))
                    .contentSecurityPolicy(csp ->
                        csp.policyDirectives("default-src 'self'; frame-ancestors 'none'"))
                    .frameOptions(fo -> fo.deny())
                    .referrerPolicy(rp -> rp.policy(
                        ReferrerPolicyHeaderWriter.ReferrerPolicy
                            .STRICT_ORIGIN_WHEN_CROSS_ORIGIN))
                )
            .cors(c -> c.configurationSource(corsSource()))
            .sessionManagement(s ->
                s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .build();
    }
}
```

Ejercicio 2: Externalización Segura de Credenciales

En este ejercicio migraremos una aplicación .NET que tiene credenciales hardcodeadas en `appsettings.json` para que use Azure Key Vault con Managed Identity. Seguiremos un proceso estructurado de tres fases.



Fase 1: Identificar

Usar `git-secrets` y `truffleHog` para localizar todos los secretos en el repositorio. Catalogar y clasificar por criticidad.



Fase 2: Migrar

Subir cada secreto a Azure Key Vault con nombre estandarizado. Configurar permisos IAM para la Managed Identity de la aplicación.



Fase 3: Integrar

Actualizar `Program.cs` para usar `AddAzureKeyVault` con `DefaultAzureCredential`. Eliminar valores del fichero de configuración.

Ejercicio 2: Antes y Después

 Antes: Credenciales expuestas

```
// appsettings.json en el repositorio
{
  "ConnectionStrings": {
    "Default": "Server=prod-db.empresa.com;
      Database=AppDB;
      User Id=admin;
      Password=Sup3rS3cr3t!"
  },
  "ApiKeys": {
    "Stripe": "sk_live_123abc...",
    "SendGrid": "SG.xxx..."
  }
}
```

 Después: Sin secretos en código

```
// appsettings.json – sólo metadata
{
  "KeyVault": {
    "Uri": "https://app-kv.vault.azure.net/"
  }
}

// Program.cs
builder.Configuration
    .AddAzureKeyVault(
        new Uri(builder.Configuration["KeyVault:Uri"]),
        new DefaultAzureCredential()
    );
```

Ejercicio 3: Vault con Secretos Dinámicos en Spring

Configuraremos Spring Boot para obtener credenciales de PostgreSQL dinámicas desde HashiCorp Vault en un entorno Docker Compose local. Cada instancia de la aplicación obtendrá credenciales únicas con TTL de 1 hora.

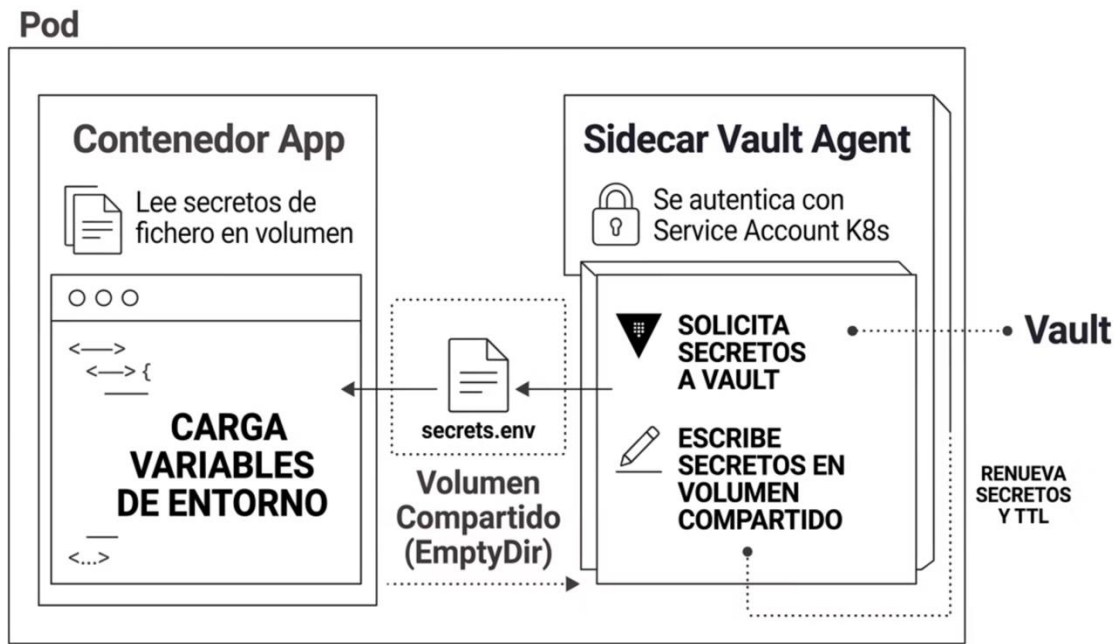
```
# docker-compose.yml
services:
  vault:
    image: hashicorp/vault:1.15
    environment:
      VAULT_DEV_ROOT_TOKEN_ID: "dev-token"
      VAULT_DEV_LISTEN_ADDRESS: "0.0.0.0:8200"
    ports: ["8200:8200"]
    cap_add: [IPC_LOCK]

  postgres:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: vault-admin-pass
      POSTGRES_USER: vault-admin
    ports: ["5432:5432"]

  app:
    build: .
    environment:
      VAULT_ROLE_ID: ${VAULT_ROLE_ID}
      VAULT_SECRET_ID: ${VAULT_SECRET_ID}
      VAULT_URI: http://vault:8200
    depends_on: [vault, postgres]
```

Patrones Avanzados: Sidecar Vault Agent

En entornos Kubernetes, el patrón **Vault Agent Sidecar** permite inyectar secretos como ficheros en el pod sin modificar el código de la aplicación. El agente se autentica con Vault usando la Service Account de Kubernetes y renueva los secretos automáticamente.



Checklist de Seguridad: Antes de Producción

Utiliza este checklist como gate de seguridad en tu pipeline CI/CD antes de desplegar a producción.

APIs

- ☐ TLS 1.2+ habilitado, 1.0/1.1 deshabilitado
- ☐ HSTS configurado con preload
- ☐ CSP restrictivo sin unsafe-inline
- ☐ Rate limiting activo en todos los endpoints
- ☐ CORS configurado con orígenes explícitos
- ☐ X-Frame-Options: DENY
- ☐ Scan con OWASP ZAP o Burp Suite

Secretos

- ☐ Cero credenciales en repositorios Git
- ☐ Pre-commit hook con git-secrets o detect-secrets
- ☐ Secretos en gestor externo (Vault / AWS / Azure)
- ☐ Rotación automática configurada
- ☐ Managed Identity o AppRole (sin contraseñas humanas)
- ☐ Auditoría de accesos activada
- ☐ Alertas de acceso anómalo configuradas

Vulnerabilidades Relacionadas a Tener en Cuenta

Las técnicas de esta sesión mitigan directamente varias de las vulnerabilidades más críticas del ecosistema de APIs según OWASP.

OWASP API4: Unrestricted Resource Consumption

Mitigado con rate limiting y throttling. Sin estos controles, un atacante puede agotar recursos del servidor con peticiones masivas automatizadas.

OWASP API8: Security Misconfiguration

Mitigado con security headers, CORS restrictivo y TLS bien configurado. Es la categoría más prevalente en APIs reales.

CWE-798: Credenciales Hardcodeadas

Eliminado por completo con gestión externa de secretos. Uno de los errores con mayor impacto potencial en sistemas en producción.

Herramientas de Auditoría y Análisis



OWASP ZAP

Scanner DAST (Dynamic Application Security Testing) gratuito. Detecta cabeceras faltantes, CORS mal configurado, endpoints expuestos y vulnerabilidades XSS/SQLi de forma automatizada.



truffleHog / detect-secrets

Herramientas de análisis de repositorios que detectan secretos expuestos en el historial de Git. Esenciales como hook pre-commit y en pipelines CI.



SSL Labs / testssl.sh

Auditoría completa de la configuración TLS: versiones, cipher suites, cadena de certificados y vulnerabilidades conocidas (BEAST, POODLE, Heartbleed).

Recursos y Referencias

Estándares y guías

- **OWASP API Security Top 10** — owasp.org/API-Security
- **OWASP Cheat Sheet: REST Security**
- **NIST SP 800-57** — Gestión de claves criptográficas
- **CIS Benchmarks** para AWS, Azure y aplicaciones web

Documentación oficial

- HashiCorp Vault — developer.hashicorp.com/vault
- AWS Secrets Manager — docs.aws.amazon.com
- Azure Key Vault — learn.microsoft.com

Herramientas del taller

- **Bucket4j** — Rate limiting para Spring
- **Spring Cloud Vault** — Integración Vault + Spring
- **VaultSharp** — SDK Vault para .NET
- **NWebSec** — Security headers para ASP.NET
- **mkcert** — Certificados TLS locales
- **git-secrets** — Detección de secretos en commits



Conclusiones de la Sesión

Proteger una API no es una única acción sino una estrategia de capas que se complementan entre sí.

Puntos Clave para Recordar



La seguridad en APIs es multicapa

Rate limiting, CORS, TLS y security headers son complementarios. Omitir cualquiera deja una superficie de ataque abierta que las demás no cubren.



Nunca un secreto en código o configuración versionada

La externalización de secretos no es opcional en producción. Los secretos dinámicos con TTL corto son el estándar de oro del sector.



La automatización es la mejor política de seguridad

Rotación automática, renovación de leases, hooks pre-commit y gates de seguridad en CI/CD eliminan el error humano como vector de riesgo.

Próximos Pasos

Para consolidar lo aprendido y preparar la siguiente sesión del módulo:

